

Guia 1

Vertex Architectural Decisions

Version: 1.0

Status: Draft

1. Objetivo

Este documento registra as decisões arquiteturais fundamentais do Vertex.

Estas decisões representam escolhas permanentes da plataforma e servem como referência para todo o desenvolvimento futuro.

Mudanças nestas decisões devem ocorrer apenas quando houver forte justificativa técnica.

2. O Android é a base da plataforma

O Vertex é executado sobre o Android.

Ele não substitui o sistema operacional nem modifica seu funcionamento interno.

Toda integração com o Android ocorre através do Android Bridge.

3. O Seed é responsável pela plataforma

O aplicativo instalado pelo usuário contém o Seed.

O Seed é responsável por:

- inicializar o Vertex;
- gerenciar versões do Core;
- recuperação do sistema;
- atualização do Core;
- rollback;
- configurações fundamentais.

O Seed deve permanecer pequeno, simples e altamente confiável.

4. O Core é infraestrutura

O Core não implementa funcionalidades voltadas diretamente ao usuário.

Sua responsabilidade é fornecer infraestrutura para os plugins.

Sempre que possível, novas funcionalidades devem ser implementadas como plugins.

5. Todo aplicativo é um Plugin

No Vertex:

«Aplicativo = Plugin»

Não existe diferença entre um aplicativo e um plugin.

Todo software executado dentro da plataforma segue o mesmo modelo arquitetural.

6. Arquitetura baseada em Plugins

A plataforma é totalmente modular.

Plugins podem ser:

- carregados;
- descarregados;
- atualizados;
- substituídos;
- removidos.

Sem necessidade de alterar o Core.

7. Comunicação padronizada

A comunicação entre plugins ocorre apenas através dos mecanismos oficiais da plataforma.

São eles:

- Capabilities;
- Events;
- State;
- Shared Objects.

Plugins não devem acessar diretamente componentes internos de outros plugins.

8. APIs como contratos

Toda comunicação ocorre através de APIs públicas.

Não é permitido acessar implementações internas.

Isso reduz acoplamento e facilita evolução da plataforma.

9. Componentes especializados

Cada subsistema possui uma responsabilidade única.

Exemplos:

- Scheduler executa tarefas.
- Power Management controla energia.
- Resource Manager controla recursos.
- Security Framework controla segurança.
- UI Runtime renderiza a interface.

Nenhum componente deve assumir responsabilidades de outro.

10. Performance como requisito arquitetural

Desempenho não é tratado como otimização posterior.

Cada decisão arquitetural deve considerar:

- CPU;
- memória;
- bateria;
- armazenamento;
- tempo de resposta.

11. Carregamento sob demanda

Plugins e recursos devem ser carregados apenas quando necessários.

Componentes inativos podem ser:

- suspensos;
- hibernados;
- descarregados.

O objetivo é reduzir consumo de recursos.

12. Atualizações sem reinstalação

A plataforma deve permitir atualização do Core independentemente do APK.

O processo deve ser:

- seguro;
- transacional;
- reversível.

13. Isolamento de falhas

Falhas devem permanecer confinadas ao componente que as originou.

Sempre que possível, o restante do sistema deve continuar funcionando normalmente.

14. Segurança por camadas

A segurança deve existir em todos os níveis da plataforma.

Incluindo:

- plugins;
- APIs;
- comunicação;
- armazenamento;
- atualização;
- execução.

Nenhum componente recebe privilégios superiores aos necessários.

15. Interface otimizada para relógios

Toda a interface do Vertex é projetada considerando dispositivos com tela redonda.

A experiência em smartwatches é prioridade.

Interfaces herdadas de smartphones não devem servir como padrão.

16. Observabilidade nativa

Todo subsistema deve fornecer informações para:

- monitoramento;
- diagnóstico;
- análise de desempenho;
- identificação de falhas.

A observabilidade faz parte da arquitetura.

17. Evolução sem reescrita

A arquitetura deve permitir crescimento contínuo.

Novos componentes devem integrar-se ao sistema existente sem exigir grandes modificações estruturais.

18. Compatibilidade

Sempre que possível, novas versões devem preservar compatibilidade com versões anteriores.

Mudanças incompatíveis devem ser exceções cuidadosamente planejadas.

19. Aprender com projetos anteriores

As decisões arquiteturais do Vertex devem considerar a experiência acumulada pela indústria.

Sempre que possível:

- reutilizar boas soluções;
- evitar problemas já conhecidos;
- estudar limitações de outras plataformas;

- adotar práticas comprovadamente eficazes.

20. Decisões orientadas ao longo prazo

Decisões arquiteturais devem priorizar:

- simplicidade;
- manutenção;
- previsibilidade;
- escalabilidade;
- estabilidade.

Soluções temporárias que prejudiquem a arquitetura futura devem ser evitadas.

Declaração Final

As decisões arquiteturais descritas neste documento estabelecem a base permanente do Vertex.

Toda nova funcionalidade, componente ou mudança estrutural deve respeitar estas decisões, garantindo que a plataforma permaneça consistente, modular, estável e preparada para evoluir durante muitos anos.

AD-001 — Android como plataforma base

Decisão

O Vertex será executado sobre o Android e não substituirá o sistema operacional.

Motivação

aproveitar drivers e APIs existentes;
reduzir complexidade;
manter compatibilidade com dispositivos Android.

Alternativas consideradas

sistema operacional próprio;
fork do Android.

Consequências

dependência das APIs Android;
desenvolvimento mais simples;
maior compatibilidade.

AD-002 — Separação entre Seed e Core

Decisão

O Seed será independente do Core.

Motivação

Permitir atualização, recuperação e rollback do Core sem reinstalar o aplicativo.

Consequências

maior robustez;

arquitetura mais modular;
recuperação simplificada.

AD-003 — Core exclusivamente de infraestrutura

Decisão

O Core não implementará funcionalidades voltadas ao usuário.

Motivação

Reduzir o tamanho do Core e facilitar manutenção.

Consequências

Toda funcionalidade visível será implementada por plugins.

AD-004 — Aplicativos são plugins

Decisão

Todo aplicativo será um plugin.

Motivação

Modelo único de desenvolvimento.

Consequências

APIs unificadas;

ciclo de vida único;

menor complexidade arquitetural.

AD-005 — Comunicação oficial

Decisão

Toda comunicação utilizará:

Capabilities;

Events;

State;

Shared Objects.

Consequências

baixo acoplamento;

melhor isolamento;

comunicação padronizada.

AD-006 — Carregamento dinâmico

Decisão

Plugins serão carregados somente quando necessários.

Motivação

Economia de memória e bateria.

AD-007 — Componentização automática

Decisão

Plugins poderão ser divididos em componentes quando ultrapassarem métricas definidas.

Motivação

Escalabilidade.

AD-008 — ECS opcional

Decisão

O modelo Entity Component System será usado apenas quando houver ganho técnico comprovado.

AD-009 — UI própria

Decisão

O Vertex possuirá um sistema próprio de UI.

Motivação

Controle total da renderização e otimização para relógios redondos.

AD-010 — Android Bridge

Decisão

Toda interação com o Android passará pelo Android Bridge.

AD-011 — APIs como contrato

Decisão

Plugins nunca acessarão implementações internas do Core.

AD-012 — Atualização transacional

Decisão

Atualizações serão preparadas antes da ativação e poderão ser revertidas.

AD-013 — Budgets de recursos

Decisão

Todo plugin executará sob limites controlados de CPU, memória, bateria e armazenamento.

AD-014 — Segurança por padrão

Decisão

Permissões, sandbox e isolamento serão obrigatórios.

AD-015 — Observabilidade nativa

Decisão

Todos os módulos fornecerão métricas, logs e informações de diagnóstico.

AD-016 — Kotlin como linguagem principal

Decisão

Todo o desenvolvimento do Vertex será realizado prioritariamente em Kotlin.

Motivação

integração com Android;

corrotinas;

manutenção simplificada;

ecossistema maduro.

Exceções

Outras linguagens poderão ser utilizadas apenas quando houver justificativa técnica clara, como desempenho, segurança ou integração com bibliotecas específicas.

AD-017 — Arquitetura orientada à evolução

Decisão

Toda nova funcionalidade deve ser projetada para evoluir sem exigir reestruturações significativas da plataforma.